View Coherence under Mutable Aliasing: Writes through Multiple Types of One Object

Deep Dive H for "Refinement and Inference Growth in SKJS"

Models revision `bafdcf7c` of `main` (2026-08-05)

Abstract

The main paper's membership semantics is get-only and snapshot; its Write Authorization definition (registry row **SET-SEM**) gives each type of an object a second component: the writes accepted by its `set`-types. Row **L-MUT-ALIAS** requires a write accepted through one type to preserve the membership guarantees of every other live type of the same runtime object. We examine how those multiple types arise: subtyping, assignment after narrowing, and types introduced directly by a refinement rule. Every remaining construction path is assigned to one of these cases or placed explicitly outside the model.

Two conditions together guarantee coherence. Subtyping must not increase write capability (**L-SUB-FIELDS-SET**), and a source type must accept only writes that remain within its own readable type. The checker enforces the first condition, but narrowing can break the second. Ordinary declaration and subtyping paths reject the problematic field, array, and override conversions, often more strictly than tsc. Types introduced by `instanceof` narrowing are coherent because the implementation uses rigid type arguments, preserves already-compatible branches, and retains readonly members. The witnessed failure occurs when a previously narrowed object is assigned to a narrower alias: a legal write through either original reference can invalidate the other reference's type. The intersection operation also unions accepted write types, but no independent program path to an incoherent live pair was found. The paper records repair requirements for each failing case.

Contents

1 Purpose and Ledger Rows Addressed **2**

2 Write Permissions **2**

3 The Coherence Property and Its Two Mechanisms **2**

4 Site Audit **3**

4.1 Installation inventory 3
4.2 Subtyping checks for declaration-derived views 4
4.3 Aliases created after field narrowing: refuted by witnesses 4
4.4 Views created by `instanceof`, coherent by rigid instantiation 5
4.5 The intersection's `set`-union: mechanism without a found route 7
4.6 Read-path merge, accessors 8

5 Status and Registry **8**

6 Implementation Anchor Index **10**

1 Purpose and Ledger Rows Addressed

This document is deep dive H in the series supporting the main paper (*Refinement and Inference Growth in SKJS*). It owns the registry row **L-MUT-ALIAS** (cross-view mutation coherence). Every other row this paper names is an import; the authoritative list, with each row's canonical statement, owner, and status, is the generated review contract in Section 5 — citations in the body are checked against it.

The division of labor with deep dive F is by axis. F is temporal: does an installed view *stay* valid across mutation events (invalidation policy, the W-Field re-widening rule)? This paper is spatial: is the *set of simultaneously live views* of one object coherent at the moment a view is created? The two compose into the write-preservation clause of an eventual heap-level soundness statement: this paper for coherence at creation, F for preservation afterwards. That composition is a later heap-level proof obligation, not a fact assumed here: every transition analyzed by F must be shown to preserve this paper's relational coherence invariant, and aliases created before a narrowing involve both obligations.

impl: `TTypeSubtyping.sk / TFields::isSub_; TypeCheckRefine.sk / narrowField, evalInstanceof; TType.sk / TFields::resolveFieldType`

2 Write Permissions

We use the main paper's Write Authorization definition unchanged (registry row **SET-SEM**): a view b authorizes the write of v at key p iff $v \in [\![\mathsf{setty}(b,p)]\!]$, where $\mathsf{setty}(b,p)$ resolves to the intersected set-types of a named field, else the covering index's set-type, and readonly ($\mathsf{set} = \bot$) authorizes nothing. The checker realizes the definition at assignment sites by checking the right-hand side against setty with `isSub` — so every verdict below is conditional on **L-SUB** exactly where a subtype check decides it. Writes mediated by method calls (a setter-style method, `push`) are split into this paper's open row **L-MUT-ALIAS-CALL**: the signature semantics **E-SIG**, as stated, concerns behavioral call soundness and does not by itself guarantee preservation of aliases after receiver mutation, so the question cannot be assigned a status until a receiver-effect strengthening exists — the alias pair from Section 4.3 reaches every alias through a setter-style method exactly as through the direct write analyzed there.

Write $\mathsf{getty}(b,p)$ for the resolved get-type ($f.\mathsf{get}$, or the covering index's). Both resolvers are total: $\mathsf{setty}(b,p) = \bot$ when no writable field or index covers p (the view authorizes nothing there), and $\mathsf{getty}(b,p) = \top$ when b does not constrain p (the extensible semantics constrains mentioned keys only). Optionality does not enter: an accepted write makes the key present, and the get-clause then applies. Where several constraints apply, both resolve exactly as the implementation's lvalue read does. Throughout, a *view* of a runtime object o is an object branch recorded in some scope entry (a local's flow type, a field's type) through which checked code can reach o; a view is *live* while such an entry exists.

3 The Coherence Property and Its Two Mechanisms

Definition 1 (Cross-View Coherence (registry row L-MUT-ALIAS))**.** *A set V of live views of the same runtime object is* coherent *at key p iff every write authorized by any $v_1 \in V$ preserves the membership clause of every $v_2 \in V$ at p: $[\![\mathsf{setty}(v_1,p)]\!] \subseteq [\![\mathsf{getty}(v_2,p)]\!]$ for all $v_1, v_2 \in V$.*

Definition 2 (Write-Self-Consistency)**.** *A view v is* write-self-consistent *at p iff* $[\![\mathsf{setty}(v,p)]\!] \subseteq [\![\mathsf{getty}(v,p)]\!]$ *— the writes it authorizes keep its own reads true. A declared data field has* $\mathsf{set} = \mathsf{get}$ *and is self-consistent by construction; a readonly field is self-consistent vacuously.*

Lemma 3 (Coherence Along Admission Edges)**.** *Let view y be admitted from view x by an accepted* `isSub` *(assignment, argument passing, return). If x is write-self-consistent at p, then $\{x, y\}$ is coherent at p, conditional on the accepted subtype checks (**L-SUB**) and the field-arm capability preservation (**L-SUB-FIELDS-SET**).*

Proof. Acceptance gives $[\![\mathsf{getty}(x,p)]\!] \subseteq [\![\mathsf{getty}(y,p)]\!]$ (covariant get check) and $[\![\mathsf{setty}(y,p)]\!] \subseteq [\![\mathsf{setty}(x,p)]\!]$ (contravariant set check, row L-SUB-FIELDS-SET; the readonly check is its $\bot$ case). A write through y: $v \in [\![\mathsf{setty}(y,p)]\!] \subseteq [\![\mathsf{setty}(x,p)]\!] \subseteq [\![\mathsf{getty}(x,p)]\!]$ (self-consistency of x) $\subseteq [\![\mathsf{getty}(y,p)]\!]$ — both views' clauses hold. A write through x: $v \in [\![\mathsf{setty}(x,p)]\!] \subseteq [\![\mathsf{getty}(x,p)]\!] \subseteq [\![\mathsf{getty}(y,p)]\!]$ — likewise. Coherence extends along chains of admissions from one self-consistent root by the same computation. Where p resolves through an index rather than a named field, the appeal is to the index arms' get/set checks; deep dive D records their set-direction as conditional exactly like the named-field arms', and the index-signature control probe of Section 4.2 pins the contravariant rejection empirically. □

Corollary 4 (Declaration-Only Aliasing is Coherent)**.** *If every view of o arises from declared object types by admission chains, coherence holds at every data-field key: declared fields are self-consistent (Definition 2), and Lemma 3 covers each edge. (Accessor fields are the separate row **L-GETSET**.)*

So there are exactly two ways to break coherence, and they organize every site audited below: give a program a view whose source is *not* self-consistent (Section 4.3 — refinement narrows `get` only, so a narrowed view has $\mathsf{set} \not\subseteq \mathsf{get}$), or give it a view that never passed a subtyping check at all (Section 4.4 — views created by refinement, whose coherence is the refinement rule's own obligation; the pin's `instanceof` rule discharges it by rigid instantiation).

The same split re-derives deep dive F's #598 with no new machinery: a write *through the narrowed view itself* is authorized by the wide `setty` and lands outside the narrowed `getty` — self-inconsistency exploited through one view. Deep dive F specifies a re-widening rule (W-Field) for the written view. Because that rule updates only the written view, it cannot restore a type guarantee held by another alias.

4 Site Audit

4.1 Installation inventory

The audit covers three view-creation families; the remaining ways a program comes to hold a view reduce to the following cases. *Admission* (Section 4.2): local initialization and re-assignment, argument passing, return passing, storing a reference into an object field, and array/tuple element storage — each checks the incoming value against the declared target type with `isSub` at its `TypeCheckExpr` site, so each is an admission edge. In particular, by imported **L-SUB-CLOSED-ABSENCE**, subtyping cannot create a `Closed<T>` absence fact from an extensible source: the target's concrete never-valued index rejects a source that does not already prove the same absence. *Refinement installation* (Sections 4.3 and 4.4): the R-rules. *No aliasing created*: destructuring and spread copy values into fresh objects; an object literal checked against a contextual type is a single view at birth, aliased only through later admissions. *Same entry, other axis*: closure capture shares the captured entry rather than creating a view — deep dive F's territory. *Class construction*

and inheritance: field initialization is admission against the declared field type, and the override check is confirmed by the probe of Section 4.2. *Generic instantiation*: a call to a generic function admits arguments against the instantiated signature — an admission edge like any other. *Types that authorize no writes*: enum objects (every field readonly) and `Closed<T>` absence facts are analyzed in Section 4.4's closing paragraph. *Trust boundary*: ambient (stub-declared) values enter with their declared types unchecked — part of the corpus-wide stub trust, like every other ambient fact, and the one entry point for declared intersection views (Section 4.5). The funneling of the admission families to `isSub` is an implementation claim reviewers should check at the named sites; it is asserted from those call sites, not proved.

impl: `TypeCheckExpr.sk / assign, call/return checking`

4.2 Subtyping checks for declaration-derived views

Three control probes against the pinned checker (`target/host/dev/skjs`, via `make check-file-in-docker`) confirm Corollary 4's premises are enforced:

- *Covariant field upcast, rejected.* {`f: number`} into {`f: number | string`}: the contravariant set check fails ("Type string ... is not assignable to type number"). tsc accepts this (property covariance); SKJS is stricter, trading incompleteness for the coherent pair.
- *Mutable-array upcast, rejected.* `Array<number>` into `Array<unknown>` fails the contravariant parameter check on methods. tsc accepts (method bivariance).
- *Narrowing subclass override, rejected.* `class E extends D` redeclaring `f: number` over `f: number | string` is rejected ("incorrectly extends its base class"). tsc accepts; the override would create an incoherent pair at every `E`-to-`D` upcast.
- *Index-signature admission, rejected.* {`f: number`} into {`[k: string]: number | string`} fails the same contravariant check on the index's set-type — the check covers `setty`'s index resolution, not only named fields.

Verdict: for declaration-derived views, the subtyping checks establish their half of coherence — conditional on **L-SUB**'s open cases, exactly as row L-SUB-FIELDS-SET records.

impl: `TTypeSubtyping.sk / TFields::isSub_ (shared/left/right arms); TypeCheckClass.sk (override checking)`

4.3 Aliases created after field narrowing: refuted by witnesses

Field narrowing updates the get-type only; the set-type keeps its declared width (the same fact deep dive F pins for #598). The narrowed view is therefore not write-self-consistent, and Lemma 3's hypothesis fails exactly where subtyping cannot detect it: admission checks the covariant get against the *narrowed* get and the contravariant set against the *declared* set, so both pass.

Finding 5 (An alias created from a get-narrowed source is incoherent)**.** *A get-narrowed object passes the subtyping check, and the resulting pair fails Definition 1 for writes through either alias. A write legal for the source's declared* `set`*-type falsifies the new alias's get clause — the alias need not even be writable (a readonly target goes stale the same way). Conversely, a write legal for the new alias — a same-type alias suffices — falsifies the source's own narrowing. Executed witnesses for both directions, the readonly variant, and the tsc comparison: issue report* ***`issues/narrowed-alias-stale-write.md`****. A repair applied when the alias is created must restore the full symmetric*

coherence of Definition 1, not one consequence of the source's self-inconsistency: for a source field whose get-type sits strictly below its set-type, require both *cross containments — source set below target get, and target set below source get (vacuous for a readonly target). Self-consistent declared sources satisfy both through the existing subtyping checks, so declared programs are unaffected; a one-sided repair leaves one write direction exposed.*

impl: `TypeCheckRefine.sk / narrowField (withGet); TTypeSubtyping.sk / TFields::isSub_ (shared arm); TType.sk / TFields::resolveFieldType (isLValue)`

4.4 Views created by `instanceof`, coherent by rigid instantiation

A view created by refinement is installed without any `isSub` admission, so Lemma 3 does not apply at all; coherence is the refinement rule's own obligation. The relevant rules are `instanceof` narrowing (`evalInstanceof`, `src/TypeCheckRefine.sk:362-440`) — `instanceof Array` is the same rule at the `Array` class, with no special case — and `Array.isArray`'s readonly view, the vacuous case analyzed in the `Array` paragraph below.

How the view is created. The rule instantiates the class's hidden type arguments at *skolems*: synthetic, rigid type parameters named for their provenance (`Box#T`), carrying the declared upper bound, created with collision-guarded identities (`makeSkolemTArgsOf`, `src/TEnv.sk:780-808`; `genSkolemTParam`, `src/TTypeSubst.sk:262-281`). A skolem is not an inference candidate — no write can *bind* it; each one denotes the runtime object's actual hidden argument by the imported interpretation row **SKOLEM-SEM**, and the installed view contains the object by **L-INST**. The view is then computed per branch of the scrutinee:

- *Branches already satisfying the test are kept verbatim.* A preservation probe — the class at fresh inference arguments, created in a temporary clone so nothing leaks into the real substitution (`src/TypeCheckRefine.sk:373-379`; clones cannot create skolems, `src/TTypeSubst.sk:163`) — is matched against each branch, and a matching branch is pushed unchanged (`src/TypeCheckRefine.sk:395-404`). No new view exists at all: the branch *is* the pre-narrowing view, with its own precise arguments.

- *Non-matching branches are intersected with the skolem view*, and the scrutinee's readonly members are re-imposed on the result: `keepReadonlyFrom` forces `set` $= \bot$ back onto every named field, symbol field, and index the branch declared readonly (`src/TypeCheckRefine.sk:407-416`; `src/TType.sk:2489-2512`). Its lookup is per-member-kind, so it restores readonly status at every key the branch named — but a named field the *intersection introduces* at a key the branch covered only by a readonly index escapes it (row **V-MINT-ROIDX**, below).

- *$A \top$-bound scrutinee gets the skolem class view itself* ($C\langle C\#\bar{T}\rangle$; `src/TypeCheckRefine.sk:430-439, 721-738`) — declared data fields at rigid arguments, `set` = `get`, write-self-consistent by construction.

Lemma 6 (`instanceof`-View Coherence)**.** *Let y be the view `instanceof` C installs from scrutinee branch x, and let A be any set of live views of the same runtime object, each reached from a write-self-consistent declared root by admission chains. Then $A \cup \{y\}$ is coherent at every data-field key that x does not cover solely by a readonly index (the excluded case is row **V-MINT-ROIDX**), conditional on **L-SUB/L-SUB-FIELDS-SET** (the chains), **L-INST** (view membership), **SKOLEM-SEM** (each skolem denotes the actual hidden argument), and **SET-SEM**.*

Proof. For a kept branch, $y = x$ and there is nothing to show. Otherwise $\mathsf{setty}(y,p) = \mathsf{setty}(x,p) \cup \mathsf{setty}(C\langle C\#\bar{T}\rangle, p)$ (`TField::inter` unions set-types, `src/TTypeInter.sk:228`), less the members `keepReadonlyFrom` returns to $\bot$ — removing authorizations is monotone-safe, so it suffices to analyze the two union arms. Let r be the object's construction view: the class's declared fields at the actual arguments $\bar{A}$, so $\mathsf{setty}(r,p) = \mathsf{getty}(r,p) = [\![T_p[\bar{A}]]\!]$ for a data field, and by **SKOLEM-SEM** each $C\#T_i$ denotes the actual argument A_i, hence $\mathsf{setty}(C\langle C\#\bar{T}\rangle, p) = [\![T_p[\bar{A}]]\!]$. Every view in $A \cup \{x\}$ sits on an admission chain from r, giving $\mathsf{setty}(v,p) \subseteq \mathsf{setty}(r,p)$ (contravariant accumulation, row L-SUB-FIELDS-SET) and $\mathsf{getty}(r,p) \subseteq \mathsf{getty}(v,p)$ (covariant accumulation) for each such v. A write of w through y's first arm, $w \in \mathsf{setty}(x,p) \subseteq \mathsf{setty}(r,p) = \mathsf{getty}(r,p)$, lands in $\mathsf{getty}(v,p)$ for every $v \in A$, in $\mathsf{getty}(x,p)$ — hence in the x-conjunct of $\mathsf{getty}(y,p)$ — and in the skolem conjunct, which *is* $\mathsf{getty}(r,p)$. A write through the second arm, $w \in [\![T_p[\bar{A}]]\!] = \mathsf{getty}(r,p)$, lands in the same three places by the same containments. A write through any $v \in A$ satisfies $w \in \mathsf{setty}(v,p) \subseteq \mathsf{getty}(r,p)$ and so preserves both conjuncts of $\mathsf{getty}(y,p)$. Where p resolves through an index in *both* x and y, the same computation runs on the index arms, whose set direction deep dive D records alongside the named-field arms'. The excluded case is precisely where the resolution *changes*: x covers p only by a readonly index while the class view contributes named p, so y resolves p through the introduced named field — whose setter is the union computed by `TField::interIndex`, absorbing the index's $\bot$ — and the transfer above does not apply (row **V-MINT-ROIDX**). □

The lemma's premise localizes what remains exposed: a scrutinee branch that was previously get-narrowed is *not* on a self-consistent admission chain, and the intersection installed by refinement keeps its set-width — so such a pair inherits the incoherence of Section 4.3's site through the same union. That is the previously established defect reached through a longer route, not a defect of the `instanceof` rule. Accessor fields are **L-GETSET**; call-mediated writes are **L-MUT-ALIAS-CALL**.

A class field covered by a readonly index (row V-MINT-ROIDX). The one write-permission defect of the `instanceof` rule itself is the resolution change excluded from the lemma. Mechanism, verified in source: `TField::interIndex` builds the introduced named field with $\mathsf{set} = \mathsf{set}_1 \cup \mathsf{set}_2$ (`src/TTypeInter.sk:245-262`), so a readonly covering index's $\bot$ setter is absorbed by the class field's writable one; and `keepReadonlyFrom` checks a named field only against the original's *named* fields (`original.maybeGet`, `src/TType.sk:2489-2512`), so a field the intersection introduced at an index-only key is never restored. The resulting view then authorizes a write the scrutinee view prohibited. Its status is VIOLATED by code-level derivation: three checked-code admission routes into a readonly-covering-index view of a class instance were probed at the pin and all rejected (a class instance into an index-signature type, directly, through an intermediate named-field binding, and through a function boundary — SKJS is stricter than tsc on each), so the remaining candidate route is the ambient/stub trust boundary. The repair is to make `keepReadonlyFrom` consult the original's covering index for introduced named fields, with the symbol kind handled identically.

impl: `TTypeInter.sk / TField::interIndex, interNamedFields; TType.sk / keepReadonlyFrom`

`Array` probes. The rule at `Array` creates `Array⟨Array#T⟩`: the index set-type is the skolem, so a write of any program-typeable value is rejected — probed at the pin: on `x: unknown`, `if (x instanceof Array) { x[0] = "s" }` is rejected ("`string` is not assignable"), and on `x: ReadonlyArray<number>` even `x[0] = 1` stays rejected, the readonly index surviving the intersection by `keepReadonlyFrom`'s index arm (`src/TType.sk:2505-2510`). The `Array.isArray` path

creates `ReadonlyArray`$\langle\top\rangle$ instead (`evalIsArray`, `src/TypeCheckRefine.sk:326-358`) — the vacuous case of Definition 2: it authorizes no writes and is self-consistent for free.
impl: `TypeCheckRefine.sk / evalInstanceof; TEnv.sk / makeSkolemTArgsOf; TType.sk / TFields::keepReadonlyFrom; TTypeSubst.sk / Subst::genSkolemTParam`

Other refinement rules. R-Eq, R-EqField, and R-IsArray install intersections of *already-live* views (the scrutinee's current type against a value type or brand) rather than views of new provenance; their write-permission behavior reduces to the intersection's set direction, audited next. R-In's true branch synthesizes a named field for the tested key, but with no new authorization: where an index covers the key, the field copies the index's get *and* set — the same writes setty's index resolution already authorized — and where none does, it is a phantom with get $= \top$, set $= \bot$, invisible to subtyping and erroring at access (`src/TypeCheckRefine.sk:555-593`). `switch` narrowing is an R-Eq client: each case label narrows the scrutinee through a synthesized strict-equality condition (`src/TypeCheckStmt.sk:610-614`), and fallthrough entries carry the falling clause's whole scope — no separate install mechanism. `Closed<T>` constrains key *absence*; unmentioned keys have setty $= \bot$, so no checked write through the closed view can add a key. By imported **L-SUB-CLOSED-ABSENCE**, admission does not create the absence fact from an extensible source, and conditional on **L-SUB**/**L-SUB-FIELDS-SET** a later admitted view acquires no write capability the closed source lacked; absence facts therefore carry no writable fields. Enum objects declare every member and reverse-mapping field readonly (`src/TypeCheckEnum.sk:555-609`) — vacuously coherent.

4.5 The intersection's set-union: mechanism without a found route

`TField::inter` *unions* the operands' set-types (deep dive B flags the direction and defers analysis of write permissions to this paper). By **L-UNION** the unioned type accepts every value either operand's set accepts — the permissive direction: a write only one operand's view authorizes passes through the merged view, and the *other* operand's get-clause has no reason to survive it. The mechanism is real. What this audit could not produce is a route to an incoherent live pair through it:

- *By declaration*: an intersection-typed target (`A & B`) has its set-types *widened* by the union, so the contravariant check rejects admission into it from either operand's type (probed at the pin: rejected). tsc, which intersects property types instead, rejects the subsequent write; SKJS rejects the admission — different mechanisms, same outcome.
- *By refinement*: the rules of Section 4.4 install intersections of the scrutinee's own views — including the skolem-view intersection, whose unioned set is exactly what Lemma 6 covers; producing a runtime value that inhabits two views with genuinely different set-types at one key requires admission or a refinement step that Sections 4.2 and 4.3–4.4 already analyze.
- *By ambient declaration*: a stub-declared intersection-typed value enters with no checked admission — the stub trust boundary of Section 4.1, not a checked-code route.

Verdict: no independent defect established; the direction stays rejected by subtyping on every route found. Because this question's status differs from the witnessed sites', it is its own open registry row **V-INTER-SET** — a sound repair for Finding 5 should keep it in view, since narrowing-installed intersections are exactly where the union becomes reachable without passing a subtyping check.
impl: `TTypeInter.sk / TField::inter (set => TType::union)`

4.6 Read-path merge, accessors

The multi-branch read merge (`TFields::union`) *intersects* set-types — the conservative dual, writes through the summary authorized only if every branch authorizes them — so the branch-merge client adds no exposure (deep dive B proves the read direction; the set direction is restrictive by construction). Accessor fields, where get and set are distinct declared members and self-consistency is not a syntactic identity, are the open row **L-GETSET** and are not analyzed here.
impl: `TTypeUnion.sk / TField::union (set => inter)`

5 Status and Registry

Verdict for L-MUT-ALIAS. The row's status is rendered from the registry below; the evidence this paper contributes decomposes site by site — one witnessed refutation, with the open questions split into their own rows:

- aliases created after narrowing: refuted, witnessed for writes through either alias (Finding 5);
- views created by `instanceof`: coherent by rigid instantiation (Lemma 6), conditional on **L-SUB**/**L-SUB-FIELDS-SET**/**L-INST** and the admission-chain premise — a previously get-narrowed scrutinee routes the alias-after-narrowing defect through the refinement, adding nothing new;
- the intersection set-union: mechanism present, no reachable incoherent pair found — the open row **V-INTER-SET** (Section 4.5);
- call-mediated mutation: cannot be assigned a status until E-SIG acquires receiver effects — the open row **L-MUT-ALIAS-CALL** (Section 2).

Declaration-only aliasing is coherent by Corollary 4, conditional on **L-SUB**/**L-SUB-FIELDS-SET**, with the four rejection probes of Section 4.2 checking the subtyping behavior (the index-signature control extending them to setty's index resolution) and the two `Array` probes of Section 4.4 checking the write permissions of the created view. The analysis of the refuted site states its fix direction.

Registry. The authoritative statuses are maintained in the registry (`docs/ledger.yaml`); this paper is cited there as evidence. The rows it supports:

Row	Statement	Status
L-MUT-ALIAS	A checked field or index write through one mutable view preserves the membership guarantees of every live alias of the same runtime object.	VIOLATED
L-MUT-ALIAS-CALL	Calls that mutate a receiver (setter-style methods, `push`) through one view preserves the membership guarantees of every live alias.	OPEN
V-INTER-SET	Reachability of an incoherent live pair through the intersection's permissive set-union, independent of the already violated subtyping and narrowing sites.	OPEN
V-MINT-ROIDX	The `instanceof` class arm grants a write at a key the scrutinee covered only by a readonly index: the intersection's named field keeps a writable setter that survives `keepReadonlyFrom`.	VIOLATED

Imports (review contract). The named statements this paper assumes from elsewhere. A reviewer treats each as an axiom, checking citations against the registry's canonical statement; no other paper's internals are in scope:

Row	Canonical statement	Owner	Status
L-UNION	`union` over-approximates set union (all levels).	B	PROVED
L-SUB	`Clauses::isSub` acceptance implies semantic inclusion under the current state.	D	OPEN
L-INST	$P_{\mathsf{inst}}(C) \subseteq [\![T_C]\!]$ for the instance type as instantiated at the narrowing site.	A	PROVED
L-MUT	Field writes are governed by `set`-types; get-only membership is sound for snapshot reads.	main	ASSUMED
L-GETSET	Get/set coherence of accessor fields.	F	OPEN
E-SIG	Behavioral call-soundness semantics for function signatures (required for function subtyping to be a soundness statement).	D	ASSUMED
SKOLEM-SEM	A skolem in an installed `instanceof` view denotes the scrutinee object's actual hidden type argument, fixed at narrowing time and stable thereafter.	main	PROVED
SET-SEM	A view authorizes exactly the writes its resolved `set`-types accept; a readonly field authorizes nothing.	main	PROVED
L-SUB-FIELDS-SET	`isSub`'s field arms preserve write capability downward: acceptance implies each target field's `set`-type is below the source's, and readonly fields acquire no writers.	D	OPEN
L-SUB-CLOSED-ABSENCE	`isSub` admission cannot create a `Closed<T>` absence fact: acceptance requires the source to prove `get` $= \bot$ at every key the target proves absent.	D	OPEN

6 Implementation Anchor Index

This document	Implementation
write resolution	`TType.sk / TFields::resolveFieldType (isLValue); TypeCheckExpr.sk / assign`
field subtyping checks	`TTypeSubtyping.sk / TFields::isSub_, isSub_NamedFields`
override checking	`TypeCheckClass.sk`
field narrowing (get-only)	`TypeCheckRefine.sk / narrowField, evalFieldEqualityType (withGet)`
views created by refinement	`TypeCheckRefine.sk / evalInstanceof, evalIsArray; TEnv.sk / makeSkolemTArgsOf; TType.sk / TFields::keepReadonlyFrom`
rigid instantiation	`TTypeSubst.sk / Subst::genSkolemTParam`
key-fact synthesis	`TypeCheckRefine.sk / evalCond (In arm, field synthesis)`
intersection set direction	`TTypeInter.sk / TField::inter (set => TType::union)`
merge set direction	`TTypeUnion.sk / TField::union (set => inter)`